LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi antar Kelas



Farrel Labib Alhaidar

103132400009

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram kelas memetakan struktur kelas dalam sistem. Setiap kelas berisi nama, daftar atribut, dan daftar metode, dengan simbol hak akses: + untuk publik, - untuk privat, # untuk terlindungi. Modul ini membahas empat relasi antarkelas menurut notasi dan tingkat keterikatannya: asosiasi, agregasi, komposisi, dan generalisasi.

- **Relasi asosiasi**
    a. Menghubungkan dua kelas terpisah yang saling berinteraksi.
    b. Notasinya garis lurus tanpa wajik.
    c. Satu kelas menyimpan referensi ke kelas lain sebagai atribut, atau keduanya saling mengenal.
    d. Contoh: Dosen membimbing Mahasiswa. Kelas Dosen menyimpan daftar mahasiswa bimbingannya.
- **Relasi agregasi**
    e. Menyatakan hubungan bagian (part) dan keseluruhan (whole), lebih erat daripada asosiasi.
    f. Notasinya garis dengan wajik kosong (white diamond) di sisi kelas keseluruhan.
    g. Kelas keseluruhan menyimpan kelas bagian sebagai atribut. Kedua objek bisa dibuat sendiri-sendiri dan hidup mandiri.
    h. Contoh: Jurusan menampung array Mahasiswa, dan objek mahasiswa tetap ada tanpa jurusan.
- **Relasi komposisi**
    i. Bentuk agregasi yang lebih ketat.
    j. Notasinya garis dengan wajik hitam (black diamond) di sisi kelas pemilik.
    k. Objek bagian tidak bisa ada tanpa objek keseluruhan. Ketika objek induk dihancurkan, seluruh bagiannya ikut hilang.
    l. Contoh: Komputer membuat CPU, Monitor, dan Mouse di dalam dirinya saat objek komputer diciptakan.

B. Guided

1. Guided1(Asosiasi)

    Dosen.java

```
package guided.guided1;


public class Dosen {
```

```
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
```

```
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");
        Dosen d = new Dosen();
        d.giveScore(m, 90);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Mahasiswa.java

```
package guided.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }
```

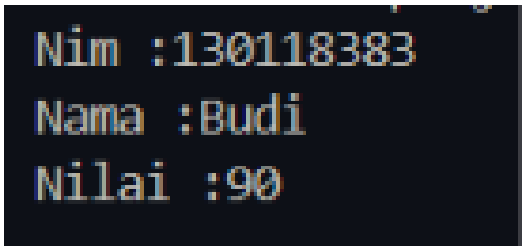
```
    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

Output :



2. Guided2 (Agregasi)

Jurusan.java

```
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan) {
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
```

```
    }

    // Behaviour
    public void addMahasiswa(Mahasiswa m) {
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++) {
            System.out.println(i + 1 + ", " +
daftarMahasiswa[i].getNIM() + " " + daftarMahasiswa[i].getNama());
        }
    }

    // Getter
    public String getKodeJurusan() {
        return kodeJurusan;
    }
    public String getNamaJurusan() {
        return namaJurusan;
    }

    // Setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }
    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
    }
}
```

Mahasiswa.java

```java
package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // Getter
    public String getNIM() {
        return NIM;
    }
    public String getNama() {
        return Nama;
    }

    // Setter
    public void setNIM(String NIM) {
        this.NIM = NIM;
    }
    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

Output :

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1, 103112400148 Rendhi
2, 103112400149 John
```

3. Guided3 (Komposisi)

Komputer.java

```
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse) {
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    // Behaviour
    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

Monitor.java

```
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand) {
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
```

```
    }
}
```

Mouse.java

```
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe) {
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec() {
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Processor.java

```
package guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    public Processor(String brand, int kecepatan) {
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    public void displaySpec() {
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Main.java

```java
package guided;

import guided.agregasi.*;
import guided.komposisi.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24
Inch", "Logitech", "Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output:

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

C. Unguided

1. Soal Unguided

perancangan sistem manajemen pesanan kafe berbasis Java untuk mempraktikkan tiga bentuk relasi antarkelas secara nyata tanpa melibatkan mekanisme pewarisan (*inheritance*). Struktur utamanya menghubungkan kelas ItemPesanan yang berelasi komposisi dengan RincianHarga, kelas Pesanan yang menampung sekumpulan ItemPesanan melalui agregasi berbasis array standar, serta kelas Kasir yang berasosiasi secara temporer dengan Pesanan saat kalkulasi transaksi dan pencetakan struk pembayaran.

Batasan Teknis: Seluruh *field* wajib berstatus private (enkapsulasi), parameter konstruktor dibuat identik dengan nama *field* (eksplisit menggunakan this), wadah data wajib memakai array konvensional bertipe tetap tanpa ArrayList, serta tidak menerapkan *inheritance* maupun *abstract class*.

Komposisi (ItemPesanan & RincianHarga): Objek RincianHarga diinstansiasi secara internal via *keyword* new langsung di dalam konstruktor ItemPesanan, serta dilarang dibuat pada *method* main.

Agregasi (Pesanan & ItemPesanan): Objek ItemPesanan dibuat secara independen di main, kemudian dimasukkan satu per satu ke dalam array penampung pada objek Pesanan melalui *method* tambahItem().

Asosiasi (Kasir & Pesanan): Kelas Kasir tidak menyimpan objek Pesanan sebagai variabel anggota permanen; interaksi antarkeduanya hanya berlangsung saat objek pesanan dikirimkan sebagai argumen parameter pada *method* prosesPembayaran().

Skenario Pengujian (Main): Alur eksekusi mencakup inisialisasi 1 Kasir, 3 objek mandiri ItemPesanan, 1 objek Pesanan meja 4, proses memasukkan item ke pesanan, penampilan detail pesanan, dan pencetakan struk pembayaran akhir.

Jawaban :

ItemPesanan.java

```
package unguided.unguided1;


public class ItemPesanan {

    private String namaItem;
```

```
    private RincianHarga rincian;


    // Constructor
    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }


    // Getter
    public String getNamaItem() {
        return this.namaItem;
    }


    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

Kasir.java

```
package unguided.unguided1;

public class Kasir {
    private String namaKasir;

    // Constructor
    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }
```

```
    // Getter
    public String getNamaKasir() {
        return this.namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();
        int jumlahItem = pesanan.getJumlahItem();

        for (int i = 0; i < jumlahItem; i++) {
            RincianHarga rincian = daftarItem[i].getRincian();
            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("\n=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlahItem);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Main.java

```
package unguided.unguided1;

public class Main {
    public static void main(String[] args) {
        // 1. Buat objek Kasir
```

```
        Kasir kasir = new Kasir("Budi");

        // 2. Buat 3 objek ItemPesanan secara independen
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Buat objek Pesanan untuk Meja 4 (kapasitas 5 item)
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Masukkan ketiga ItemPesanan ke dalam Pesanan via
tambahItem()
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan rincian pesanan
        pesanan.tampilkanPesanan();

        // 6. Proses pembayaran oleh Kasir
        kasir.prosesPembayaran(pesanan);
    }
}
```

Pesanan.java

```
package unguided.unguided1;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // Agregasi: menampung objek yang
dibuat di luar
    private int jumlahItem;

    // Constructor
    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
```

```
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga rincian = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")");
        }
    }

    // Getter
    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

RincianHarga.java

```java
package unguided.unguided1;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    // Constructor
    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10; // Pajak 10% dari harga dasar
    }

    // Getter
    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan :
Program Sistem Pesanan Cafe ini mengimplementasikan tiga relasi antarkelas dalam PBO secara terpadu. Hubungan komposisi diterapkan antara ItemPesanan dan RincianHarga di mana objek rincian harga di-instansiasi langsung di dalam constructor item pesanan sehingga siklus hidupnya terikat penuh. Sementara itu, relasi agregasi digunakan pada kelas Pesanan untuk menampung kumpulan objek ItemPesanan yang dibuat mandiri di luar dan dimasukkan melalui method tambahItem(). Untuk pemrosesan transaksi, kelas Kasir berinteraksi dengan Pesanan melalui relasi asosiasi (uses-a), di mana objek pesanan tidak disimpan sebagai atribut melainkan hanya diteruskan sebagai parameter pada method prosesPembayaran(). Melalui kelas Main seluruh alur mulai dari pembentukan item, pengelompokan pesanan meja, hingga pencetakan struk pembayaran berhasil dijalankan secara terstruktur.

D. Kesimpulan

1. Asosiasi: Relasi komunikasi antar-objek berlambang garis lurus polos, pas suatu kelas cuma berinteraksi sama objek lain lewat parameter *method* tanpa nyimpennya jadi atribut permanen.
2. Agregasi: Hubungan bagian-keseluruhan (*part-of*) bertanda wajik kosong; objek bagian disimpen jadi atribut di kelas penampung, tapi siklus hidup kedua objek tetep mandiri dan bisa dibikin terpisah.
3. Komposisi: Bentuk khusus agregasi bertanda wajik hitam solid; siklus hidup objek bagian bergantung penuh sama kelas wadahnya, jadi pas kelas utama musnah, seluruh bagian di dalamnya bakal ikut musnah.